Design and Analysis of Algorithms: CI43

Course Outline

Unit I

Asymptotic Bounds and Representation problems of Algorithms: Computational Tractability: Some Initial Attempts at Defining Efficiency, Worst-Case Running Times and Brute-Force Search, Polynomial Time as a Definition of Efficiency, Asymptotic Order of Growth: Properties of Asymptotic Growth Rates, Asymptotic Bounds for Some **Common Functions, A Survey of Common Running Times**: Linear Time, O(n log n) Time, O(nk) Time, Beyond Polynomial Time. Some Representative Problems, A First Problem: Stable Matching.

Course Outline

Unit II

Graphs & Divide and Conquer: Graph Connectivity and Graph Traversal, Breadth-First Search: Exploring a Connected Component, Depth-First Search,

Implementing Graph Traversal Using Queues and Stacks: Implementing Breadth First Search, Implementing Depth-First Search,

An Application of Breadth-First Search: The Problem, Designing the Algorithm, Directed Acyclic Graphs and Topological Ordering, The Merge sort Algorithm.

Course Outline

Unit III

Greedy Algorithms: Interval Scheduling: The Greedy Algorithm Stays Ahead: Designing a Greedy Algorithm, Analyzing the Algorithm,

Scheduling to Minimize Lateness: An Exchange Argument: The Problem, Designing the Algorithm, Designing and Analyzing the Algorithm, Shortest Paths in a Graph: The Problem, 60 Designing the Algorithm, Analyzing the Algorithm,

The Minimum Spanning Tree Problem: The Problem, Designing Algorithms, Analyzing the Algorithms, Huffman Codes and Data Compression.

Course Outline

Unit IV

Dynamic Programming: Weighted Interval Scheduling: A Recursive Procedure: Designing a Recursive Algorithm, **Subset Sums and Knapsacks**: Adding a Variable: The Problem, Designing the Algorithm, Shortest Paths in a Graph: The Problem, Designing the Algorithm, The Maximum-Flow Problem.

Course Outline

Unit V

NP and Computational Intractability: Polynomial-Time Reductions NP-Complete Problems: Circuit Satisfiability: A First NP-Complete Problem, General Strategy for Proving New Problems NPComplete, Sequencing Problems: The Traveling Salesman Problem, The Hamiltonian Cycle Problem.

Textbooks

1. Algorithm Design, Jon Kleinberg and Eva Tardos, Pearson, 1st Edition 2013.

2. Introduction to the Design & Analysis of Algorithms, Anany Levitin, 3rd Edition, 2012, Pearson education.

Reference Books

1.Introduction to Algorithms, H., Cormen, Charles E. Leiserson, Ronal L. Rivest, Clifford Stein Thomas, 3rd Edition, 2009, MIT press.

2. Fundamentals of Computer Algorithms, Horowitz E., Sartaj Sahni S., Rajasekaran S, 2008, Galgotia Publications.

Course Outcomes

At the end of the course, the student will be able to:

1. Define the basic concepts and analyze worst-case running times of algorithms using asymptotic analysis.
2. Recognize the design techniques for graph traversal using representative algorithms.
3. Identify how divide and conquer works and analyze complexity of divide and conquer methods by solving recurrence.
4. Illustrate Greedy paradigm and Dynamic programming paradigm using representative algorithms.
5. Describe the classes P, NP, and NP-Complete and be able to prove that a certain problem is NP-Complete.

Course Assessment

- CIE- 30 Marks

- Case Study-Algorithm=10 Marks
- Problem Solving=10 Marks
- Total Marks=CIE+ Assignments=50


Thank you

Unit-1

Asymptotic Bounds and Representation problems of Algorithms

Algorithm

- An **algorithm** is a **step-by-step procedure or set of rules** for solving a problem or performing a computation. It takes **input**, processes it through a sequence of well-defined steps, and produces an **output**.
- **Key Characteristics of an Algorithm**
- **Well-defined Inputs and Outputs** – An algorithm must have a clear starting point (input) and a defined result (output).
- **Definiteness** – Each step must be precise and unambiguous.
- **Finiteness** – The algorithm must terminate after a finite number of steps.
- **Effectiveness** – Each step should be simple and executable within a reasonable amount of time.
- **Generality** – The algorithm should work for a wide range of inputs, not just specific cases.

Algorithm

Example: Algorithm for Adding Two Numbers

- Problem Statement: Add two numbers and display the result.

Algorithm:

- Start
- Take two numbers as input (A and B)
- Compute the sum: Sum = A + B
- Print the result
- Stop

Types of Algorithms

Types of Algorithms

- Brute Force Algorithm – Tries all possible solutions (e.g., Linear Search).
- Divide and Conquer – Breaks the problem into smaller subproblems and solves them recursively (e.g., Merge Sort, Quick Sort).
- Greedy Algorithm – Makes the best local choice at each step (e.g., Dijkstra's Shortest Path).
- Dynamic Programming – Solves problems by breaking them into overlapping subproblems (e.g., Fibonacci using memoization).
- Backtracking – Explores possible solutions and backtracks when a solution fails (e.g., N-Queens Problem).

Algorithm Analysis

- **What is Algorithm Analysis**?

Study of an algorithm's performance in terms of time and space.

Algorithm analysis helps determine **how fast** an algorithm runs and how much **memory** it uses.

Efficiency matters—Faster algorithms save time and resources.

Scalability—Ensures performance remains stable for larger inputs.

Key Focus Areas

- **Key Focus Areas:**
- **Asymptotic Bounds** – Theoretical performance limits of algorithms.
- **Representation of Algorithms** – How we express algorithms mathematically or visually.
- **Computational Tractability** – Understanding which problems can be solved efficiently.

Efficiency

- **Early Attempts to Define Efficiency:**
- **Counting basic operations:** Initially, efficiency was measured by counting basic operations like additions or comparisons.
- **Machine-dependent measures:** Earlier, efficiency was measured based on execution time on a specific machine, but this was unreliable due to differences in hardware.
- **Need for machine-independent measures:** To generalize efficiency, researchers developed a way to measure running time as a function of input size, leading to complexity analysis.

Worst -Case Running Time and Brute-Force Search

The **worst-case running time** of an algorithm is the maximum time it takes for any input of size n. It provides an **upper bound** on performance and is important for ensuring predictable behavior.

- For example:
- **Linear Search**: Worst-case time is O(n) when the target element is at the end of the list.
- **Binary Search**: Worst-case time is O(log n) when the element is not in the list.

Brute-Force Search

- Brute-force algorithms try all possible solutions and are typically inefficient. Examples:
- **Traveling Salesman Problem (TSP)**: Checking all possible routes requires **factorial time** O(n!).
- **String Matching**: Checking every position of a text for a pattern requires **quadratic time** O(nm).

Polynomial Time as a Definition of Efficiency

- **Why Polynomial Time?**
- An algorithm is considered **efficient** if its running time is **polynomial** in the input size, i.e., it runs in:

$$O(n^c))$$

- for some constant c.

Polynomial-time algorithms grow **at a reasonable rate** and include:
Sorting algorithms: Merge Sort (O(nlog n), QuickSort (O(nlogn))
Graph algorithms: Dijkstra's Algorithm (O(n2)
Matrix multiplication: $O(n^3)$

Non-Polynomial Time Algorithms (Exponential & Factorial)
Exponential time ($O(2^n)$): Subset-sum problem
Factorial time (O(n!): Brute-force solutions in NP-hard problems

- Polynomial-time problems are classified as **P (polynomial-time solvable)**, while harder problems fall under **NP (nondeterministic polynomial time)**.

Asymptotic Bounds – Introduction

What are Asymptotic Bounds?

- **Definition:** A method of describing an algorithm's performance **as input size grows**.
- Used to compare algorithms based on **time complexity** and **space complexity**.

Why are Asymptotic Bounds Important?

- Helps analyze the **worst-case**, **best-case**, and **average-case** performance of an algorithm.
- Determines if an algorithm **scales well** with large input sizes.

Properties of Asymptotic Growth Rates

Asymptotic notation describes how an algorithm scales with input size

- **1. Big-O Notation (O) – Upper Bound (Worst Case)**

Represents the worst-case time complexity.

Example: **Bubble Sort – $O(n^2)$**

Graphical representation *(Include a chart showing $O(n)$, $O(n^2)$, $O(\log n)$, etc.)*

- **2. Big-Ω Notation (Ω) – Lower Bound (Best Case)**

Represents the best possible performance.

Example: **Insertion Sort – Ω(n) (Best case when already sorted)**

- **3. Big-Θ Notation (Θ) – Tight Bound (Average Case)**

Represents both upper and lower bounds.

Example: **Merge Sort – Θ(n log n)**

Comparing Growth Rates

- **Comparing Growth Rates**
- The order of growth (from smallest to largest):
- O(1)<O(logn) <O(n)<O(nlogn) <O(n2)<O(2n) <O(n!)
- Example: Binary Search (O(logn) is much faster than Linear Search (O(n)).
- it's **comparing upper bounds** on algorithm growth. This is why it specifically uses Big-O notation rather than Ω\OmegaΩ (best-case lower bound) or Θ\ThetaΘ (tight bound).

Representation Problems of Algorithms

- **Different Ways to Represent an Algorithm**
- **Pseudocode** – A structured stepwise description of an algorithm.
- **Flowcharts** – A graphical representation using symbols.
- **Recurrence Relations** – A mathematical formula that expresses running time.

Linear Search Algorithm

1. Start
2. Initialize an index variable i = 0.
3. Repeat while i < n (where n is the number of elements in the array):
   - Check if arr[i] == key:
     - If yes, return i (index of the found element).
     - If no, move to the next index (i = i + 1).
4. If the loop ends and the key is not found, return -1 (indicating not found).
5. Stop

Linear Search Representation

- Linear search compares each element in the list sequentially with the key value.
- **Steps to search for 74 in the given list:**
- **List:** [10, 92, 38, 74, 56, 19, 82, 37]
  Key = 74

Linear Search Representation

Step	Element Checked	Match Found?
1	10	❌ No
2	92	❌ No
3	38	❌ No
4	74	✅ Yes

Linear Search Representation

Index position of 74 = 3 (0-based indexing)
Number of comparisons = 4

- **Time Complexity Analysis**
- **Best Case (Ω(1)):**

The key is found in the first position of the array.

Example: If searching for 10, it is found in one comparison.

Complexity: Ω(1)

- **Worst Case (O(n)):**
- The key is found in the **last position** or **not found at all**.
- **Example:** If searching for **37** (last element), it takes **n comparisons**.
- **Complexity: O(n)**

Linear Search Representation

1. Best Case: Ω(1)

- Scenario: The target element is at the very beginning of the list.
- Explanation:

The algorithm makes only one comparison: it checks the first element, finds the target, and stops. Therefore, the best-case time complexity is constant, denoted as Ω(1)

- Example:

In the list [10, 20, 30, 40], searching for 10 finds a match immediately at index 0.

Linear Search Representation

Worst Case: O(n)

Scenario: The target element is either at the very end of the list or not present at all.

Explanation:

The algorithm may have to check every element.

If the target isn't in the list, it will do n comparisons (one for each element).

Thus, the worst-case time complexity is linear, expressed as

$O(n)$

Example:

In [10, 20, 30, 40], searching for 50 (not present) or 40 (last element) requires 4 comparisons (one per element).

Linear Search Representation

3. Average Case: Θ(n)

- **Scenario**: The target element is equally likely to be in any position.
- **Explanation**:
  - On average, you find the target around the **middle** of the list, requiring about n/2 comparisons.
  - **Θ(n)** (Big-Theta) , we ignore constant factors like 1/2, so the **average** complexity is still **Θ(n)**.

Linear Search Representation

Case	Number of Comparisons	Time Complexity
Best Case	1 (immediate match at first element)	Ω(1)
Worst Case	n (last element or not found)	O(n)
Average Case	(n+1)2≈0.5n	O(n)

Linear Search Representation

Case	Comparisons	Complexity
Best Case	1	Ω(1)
Worst Case	n	O(n)
Average Case	n/2	Θ(n)

Table

Algorithm	Best Case	Average Case	Worst Case	Best for
Linear Search	O(1)	O(n)	O(n)	Small or unordered datasets
Binary Search	O(1)	O(logn)	O(logn)	Sorted datasets
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	Nearly sorted data
Merge Sort	O(n log n)	O(n log n)	O(n log n)	Large datasets
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	General-purpose sorting
Dijkstra's Algorithm	O(1)	O((V+E)logV)	O(V2)	Shortest path problems

Binary Search Representation

- Example: Binary Search Representation

```
BinarySearch(arr, x):
    left = 0, right = arr.length - 1
    while left ≤ right:
        mid = (left + right) // 2          // Integer division to get index
        if arr[mid] == x:
            return mid                     // Element found
        else if arr[mid] < x:
            left = mid + 1                 // Search right half
        else:
            right = mid - 1                // Search left half
    return -1                              // Element not found
```

Binary Search Representation

Given sorted array

arr = [2, 5, 8, 12, 16, 23, 38, 45, 56, 72]

Let's search for x = 23.

left = 0, right = 9 (index range of the array)

mid = (0 + 9) // 2 = 4

arr[mid] = 16 (which is less than 23)

Since arr[mid] < x, search in the right half .

left = mid + 1 = 5. left=4+1=5

Binary Search Representation

Next iteration:

left = 5, right = 9

mid = (5 + 9) // 2 = 7

arr[mid] = 45 (which is greater than 23)

Since arr[mid] > x, search in the left half. right = mid − 1.

So, right=7−1=6

Next iteration:

left = 5, right = 6

mid = (5 + 6) // 2 = 5

arr[mid] = 23 (which is equal to x)

✅ Element found at index 5

Deriving the Complexity Formula

- Let's say Binary Search takes k steps to reduce the input size to 1. Mathematically, this means :$n/2^{k=1}$
- Solving for k:
- $n=2^k$
- **$K=\log_2 n$**
- Since Big-O notation ignores constants and bases of logarithms:
- O(logn)

Common Time Complexities:

- **Common Complexities:**
  - **O(1)** → Constant time (e.g., accessing an element in an array)
  - **O(n)** → Linear (e.g., single loop)
  - **$O(n^2)$** → Quadratic (e.g., two nested loops)
  - **O(log n)** → Logarithmic (e.g., Binary Search)
  - **O(n log n)** → Efficient sorting (e.g., Merge Sort)
  - **$O(2^n)$** → Exponential (e.g., Recursive Fibonacci)

Common Time Complexities Calculations:

1.O(1) ⟶ Constant Time

- ✅ **Happens when:**
- No loops, just direct operations.
- Example: Accessing an element in an array.
  - 🔹 **Shortcut:** If the number of operations **doesn't depend on n**, it's **O(1)**.

Ex:

```
#include <stdio.h>
void getFirstElement(int arr[]) {
    printf("%d", arr[0]);  // Always takes constant time
}
```

Common Time Complexities Calculations:

2.O(n) → Linear Time

✅ Happens when:

One loop running n times.

- 🔹 Shortcut: Single loop → O(n).

Ex:

```
#include <stdio.h>
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);  // Runs n times
    }
}
```

Common Time Complexities Calculations

3.O(n²) → Quadratic Time

✅ Happens when:

Two nested loops, each running n times.

- Shortcut: Nested loops → Multiply complexities (O(n) × O(n) = O(n²)).

Ex:

```
#include <stdio.h>
void printPairs(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("(%d, %d) ", arr[i], arr[j]);  // Runs n * n times
        }
    }
}
```

Common Time Complexities Calculations:

4.O(log n) → Logarithmic Time

✅ Happens when:

The input shrinks by half in each step (e.g., Binary Search).

- 🔹 Shortcut: If n is divided repeatedly → O(log n).

EX:

```
#include <stdio.h>
int binarySearch(int arr[], int left, int right, int key) {
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == key) return mid;
        else if (arr[mid] < key) left = mid + 1;
        else right = mid - 1;
```

Common Time Complexities Calculations:

5.O(n log n) → Efficient Sorting

✅ Happens when:

Sorting algorithms like Merge Sort, QuickSort.

These divide the problem (log n) and do work (n) → O(n log n).

- Shortcut: Divide & Conquer sorting → O(n log n).

EX:

```
#include <stdio.h>
void mergeSort(int arr[], int l, int r) {
    if (l >= r) return;
    int mid = l + (r - l) / 2;
    mergeSort(arr, l, mid);
    mergeSort(arr, mid + 1, r);
    // Assume merge() function exists
}
```

Common Time Complexities Calculations:

6.O(2^n) → Exponential Time

✅ Happens when:

Recursive calls double in each step (e.g., Fibonacci).

- 🔹 Shortcut: If each call makes 2 recursive calls → O(2^n).

EX:

```
#include <stdio.h>
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

Examine the time complexity of the given code snippets

```
1.int fun(int n)
{
  int count = 0;
  for(int i = 0; i < n; i++)
     for(int j = i; j > 0; j--)
        count = count + 1;
  return count;
}
```

Examine the time complexity of the given code snippets

Step-by-Step Complexity Analysis

The outer loop runs from i = 0 to n-1, i.e., n iterations.

The inner loop runs from j = i down to 1 (i.e., i times).

Total Number of iterations:

$\sum_{i=0}^{n-1}$ $i=0+1+2+3+\ldots+(n-1)=2n(n-1)=O(n^2)$

2n(n-1) or $n(n-1)/2 = O(n^2)$

Time Complexity: $O(n^2)$----------□(Nested loops with decreasing iterations)

Examine the time complexity of the given code snippets

```
2)void fun(int n, int arr[ ]) {
    int i = 0, j = 0;
    for (; i < n; ++i) // Outer loop runs n times
        while (j < n && arr[i] < arr[j]) // Inner loop condition
            j++;
}
```

Step-by-Step Complexity Analysis

The outer loop runs n times.

The inner while loop increases j, but j is not reset inside the loop.

Since j only increases and does not decrease, it does not run n times for every i.

Instead, j goes from 0 to n-1, across all iterations of i.

Total number of iterations: O(n) ------□(Single pass for j across all iterations)

Final Complexity: O(n)

Computational Tractability – Definition

- **What is Computational Tractability?**
- Computational tractability refers to whether a problem can be solved efficiently using an algorithm that runs in **polynomial time** (O(n^k) for some constant k).
- Polynomial time refers to the class of algorithms that run in **O(n^k)** time, where **n** is the input size and **k** is a constant. These algorithms are considered **efficient and feasible** for large inputs.

- **Tractable Problems**: Problems that can be solved in **polynomial time (P-class problems)**.
- **Intractable Problems**: Require **exponential or factorial time**, making them impractical for large inputs.

Examples of Tractable & Intractable Problems

Problem	Time Complexity	Tractability
Sorting (Merge Sort)	O(n log n)	✅ Tractable
Matrix Multiplication	$O(n^3)$	✅ Tractable
Graph Coloring	$O(2^n)$	❌ Intractable
Traveling Salesman (Brute Force)	O(n!)	❌ Intractable

A Survey of Common Running Times

- In computational complexity, algorithms have different running times based on how their execution grows with input size **n**. Below is an overview of common time complexities:

• O(1)	Constant	Accessing an array element
• O(log n)	Logarithmic	Binary Search
• O(n)	Linear	Finding max in an array
• O(n log n)	Near Linear	Merge Sort, Quick Sort
• $O(n^2)$	Quadratic	Bubble Sort, Floyd-Warshall
• $O(n^3)$	Cubic	Matrix Multiplication
• $O(2^n)$	Exponential	Recursive Fibonacci
• O(n!)	Factorial	Traveling Salesman Problem

$O(n^k)$ Time Complexity (Polynomial Time)

- **1. $O(n^k)$ Time Complexity (Polynomial Time)**
- **Definition**: Any algorithm where the number of operations is proportional to **n raised to a constant power (k)**.
- **Examples**:
  - Matrix multiplication (**$O(n^3)$**)
  - Floyd-Warshall algorithm (**$O(n^3)$**)
  - Dynamic programming solutions for problems like **Longest Common Subsequence ($O(n^2)$).**
  - **The above are Tractable Problems**

Beyond Polynomial Time (Intractable Problems)

Factorial Time (O(n!)):

Definition: The number of operations is proportional to n! (factorial).

Examples:

Traveling Salesman Problem (TSP) using brute force

Generating all permutations of a string

Brute Force TSP (O(n!))

```
int tsp(int graph[][N], int v[], int n, int pos, int count, int cost, int minCost) {
    if (count == n && graph[pos][0]) {
        return cost + graph[pos][0];
    }
    for (int i = 0; i < n; i++) {
        if (!v[i]) {
            v[i] = 1;
            minCost = tsp(graph, v, n, i, count + 1, cost + graph[pos][i], minCost);
            v[i] = 0;
        }
    }
    return minCost;
}
```

Recurrence Relations Substitution Method

Solve the following recurrence relations using substitution method

i. $x(n)=x(n-1)+5$ for $n>1, x(1)=0$

(i) $x(n)=x(n-1)+5x(n) =0$

$x(2)=x(1)+5=0+5=5$

$x(3)=x(2)+5=5+5=10$

$x(4)=x(3)+5=10+5=15$

- Observing the pattern, we can generalize:

$x(n)=x(1)+5(n-1) =0+5(n-1)=5(n-1)$

Correct formula: $x(n)=5(n-1)$

The Gale-Shapley algorithm

1. **Each proposer proposes to their most preferred receiver who has not yet rejected them.**
2. **Each receiver evaluates the proposals and:**
   - **Accepts the most preferred proposer (tentatively).**
   - **Rejects the others.**
3. **Rejected proposers move to their next preferred choice.**
4. **The process repeats until all proposers are matched.**

- **Time Complexity:**
- In the worst case, each proposer can propose to every receiver, leading to O(n2) iterations.
- Since each proposal is made at most once, the overall complexity is **O(n2)**, where n is the number of proposers (or receivers, assuming they are equal).

The Gale-Shapley algorithm

The Gale-Shapley algorithm, also called the Stable Matching Algorithm, is used for solving the stable marriage problem.

it applies to many real-world scenarios, such as job hiring, student admissions.

- **Example: Stable Matching in University Admissions**
- We have **3 students** and **3 universities**, each with ranked preferences.

Step 1: Given Preferences

- **Students' Preferences:**
- **S1:** U1 > U2 > U3
- **S2:** U2 > U1 > U3
- **S3:** U1 > U3 > U2

Universities' Preferences:

- **U1:** S2 > S1 > S3
- **U2:** S1 > S2 > S3
- **U3:** S1 > S2 > S3

The Gale-Shapley algorithm

Step 2: Apply Gale-Shapley Algorithm

- Each **student applies** to their **top-choice university**.
- Each **university tentatively accepts** the best student while rejecting others.
- **Rejected students apply to their next preferred university.**
- The process continues until all students are placed.

The algorithm runs in $O(n^2)$ time complexity, where n is the number of participants in one group (e.g., students or universities).

In the worst case, each student may have to propose to all universities before getting a stable match. Since there are n students and each may apply to n universities, the worst-case time complexity is $O(n^2)$.

The Gale-Shapley algorithm

Step	Proposals	Tentative Matches
1	S1 → U1, S2 → U2, S3 → U1	U1(S1), U2(S2)
2	S3 (rejected by U1) → U3	U1(S1), U2(S2), U3(S3)

The Gale-Shapley algorithm

- Since all students are matched, the process stops.
- **Final Stable Matching**
- ✅ **(S1, U1), (S2, U2), (S3, U3).**

This algorithm can be used to solve any stable matching problems

The Gale-Shapley algorithm

Example:Consider 3 men and 3 women with preference lists:

Men:

M1: W1 > W2 > W3

M2: W2 > W3 > W1

M3: W3 > W1 > W2

Women:

W1: M2 > M1 > M3

W2: M1 > M3 > M2

W3: M3 > M2 > M1

The **Gale-Shapley algorithm** ensures that everyone is matched optimally without any unpaired individuals.

The Gale-Shapley algorithm

(ii) Blocking Pair

A **Blocking Pair** occurs when:

- A man and a woman prefer each other more than their current partners.
- This creates instability in the matching.

Example:

- If M1 is paired with W3 but M1 prefers W2,
- and W2 prefers M1 over her current partner,
- then (M1, W2)
- forms a blocking pair, making the current matching unstable.

The Gale-Shapley algorithm

(iii) Man-Optimal Matching

- A **Man-Optimal Matching** means:
- Each man gets the **best possible partner** according to his preference list.
- The algorithm guarantees that **no man can get a better partner by switching**.
- In the **Gale-Shapley algorithm**, if **men propose**, they end up with their best possible partners under stable matching.

The Gale-Shapley algorithm

(iv) Woman-Optimal Matching

- A **Woman-Optimal Matching** means:
- Each woman gets the **best possible partner** according to her preference list.
- The algorithm guarantees that **no woman can get a better partner by switching**.
- If **women propose instead of men**, they will get the best possible partners under a stable matching.
- The **time complexity** of the Gale-Shapley algorithm is **O(n2)** in the worst case. Here's why:
- **Proposals:** In the worst-case scenario, each of the n proposers (e.g., students or men) may need to propose to every one of the ns receivers (e.g., universities or women) before finding a stable match.
- **Iterations:** Since each proposal is processed only once, the total number of proposals is at most n×n, leading to O(n2) iterations.


Thank
You